

LAB MANUAL

LAB 1, 2 & 3 : File System Analysis for Mobile Device Forensics

(6 Hours)

Tools, Installation, and Command-Line Practical (Linux & Windows)

Course: Mobile Devices Forensics (SEC4336-3)
Maps to Course Content Topics 3 (Forensic Tools) and 5 (Analysis of Evidence)

Version 1.0

1. Lab Overview

This lab teaches file-system analysis as applied to a mobile-device extraction: how to examine a partition layout, inspect the file system it contains, list active and deleted files, extract individual files by inode, and analyze the SQLite databases and app data that hold the actual evidence (messages, contacts, timestamps). All tools and commands are demonstrated on both Ubuntu Linux and Windows so that students can work in whichever environment they have available.

The lab uses a self-generated, legally unambiguous evidence set: a logical file-system extraction pulled from the Android emulator (AVD) set up in the “Mobile Device Fundamentals” lab, via adb pull. This avoids any question about legal authority to analyze a real device while still producing a realistic Android /data directory structure, SQLite databases, and shared-preferences/XML files to work with.

Course Context

Course Code	SEC4336-3
Lab Topic	File System Analysis (command-line + GUI tools)
Sessions	3 sessions (2 hours each — 6 contact hours total)
Prerequisite	Completion of the “Mobile Device Fundamentals” introductory lab (Android emulator + ADB basics)
Aligned CLOs	2.1, 2.2, 2.3 (Skills)
Platforms Covered	Ubuntu Linux 22.04 LTS and Windows 10/11

2. Learning Objectives

By the end of this lab, students will be able to:

1. Install and configure The Sleuth Kit, Autopsy, DB Browser for SQLite, ADB, and ALEAPP on both Linux and Windows.
2. Pull a logical file-system extraction from an Android device/emulator using ADB.
3. Use Sleuth Kit command-line tools to examine partition tables, file-system metadata, and file listings (including deleted entries).
4. Extract a specific file from a raw image by inode number and verify it with a hash.
5. Use Autopsy to build a case, ingest a data source, and review automatically extracted artifacts.
6. Query recovered SQLite databases directly to reconstruct message and contact records.
7. Run ALEAPP to auto-parse an extraction and interpret its HTML report.

3. Tools Used — Summary

Tool	Platform	Purpose
The Sleuth Kit (TSK)	Linux, Windows	Command-line suite for partition, file-system, and file-level forensic analysis (mmls, fsstat, fls, istat, icat).

Autopsy	Windows (primary)	GUI front-end built on The Sleuth Kit; case management, ingest modules, keyword search, reporting.
DB Browser for SQLite	Linux, Windows	Opens and queries SQLite databases (contacts2.db, sms.db, app data) recovered from a device.
Android Platform-Tools (ADB)	Linux, Windows	Pulls a logical file-system extraction from the Android emulator used as our practice “device.”
ALEAPP	Linux, Windows	Python-based Android Logs Events And Protobuf Parser — automatically parses a file-system extraction into an HTML report.
ExifTool	Linux, Windows	Reads embedded metadata (timestamps, GPS, device model) from recovered media files.

Evidence & Legal Reminder: Every command in this lab is run only against your own emulator extraction or a publicly licensed practice image (Section 4.2). Never point these tools at a real person's device without documented legal authority and a proper chain of custody — covered in Topic 2 of this course.

4. Session 1 — Environment Setup (Linux & Windows)

Duration: 2 hours

4.1 Linux (Ubuntu 22.04 LTS) Setup

Open a terminal and run the following, in order.

Step 1 — Update the system

```
sudo apt update && sudo apt upgrade -y
```

Step 2 — Install The Sleuth Kit

```
sudo apt install -y sleuthkit
```

Verify installation:

```
tsk_version  
mmls -V
```

Step 3 — Install DB Browser for SQLite

```
sudo apt install -y sqlitebrowser
```

Step 4 — Install Android Platform-Tools (ADB)

```
sudo apt install -y android-tools-adb android-tools-fastboot  
adb version
```

Step 5 — Install Python 3, pip, and Git (needed for ALEAPP)

```
sudo apt install -y python3 python3-pip git
```

Step 6 — Install ALEAPP

```
git clone https://github.com/abrignoni/ALEAPP.git  
cd ALEAPP  
python3 -m pip install -r requirements.txt
```

Step 7 — Install ExifTool

```
sudo apt install -y libimage-exiftool-perl  
exiftool -ver
```

Note on Autopsy (Linux): Basis Technology distributes Autopsy's GUI installer for Windows. On Linux, Autopsy can be built from source following the instructions in its GitHub repository (github.com/sleuthkit/autopsy), but this is time-consuming and version-sensitive. For this lab, Linux users perform the file-system analysis in Section 5 using the Sleuth Kit command-line tools installed above (they use the same engine Autopsy runs internally) and switch to the Autopsy GUI walkthrough in Section 6 on a Windows machine or the lab's Windows workstations.

4.2 Windows 10/11 Setup

Step 1 — Install Autopsy (includes The Sleuth Kit)

- Go to autopsy.com → Downloads and download the current Windows installer (.msi).
- Run the installer with default options. This also installs the Sleuth Kit command-line tools under the Autopsy installation's "sleuthkit" subfolder (e.g., C:\Program Files\Autopsy-<version>\sleuthkit-<version>\bin).

- Add that bin folder to your PATH environment variable so mmls, fls, icat, etc. can be run from any Command Prompt / PowerShell window.

Step 2 — Install DB Browser for SQLite

- Download the Windows installer from sqlitebrowser.org → Downloads.
- Run the .exe installer with default options.

Step 3 — Install Android Platform-Tools (ADB)

- Download “SDK Platform-Tools for Windows” from developer.android.com/tools/releases/platform-tools.
- Extract the ZIP to a permanent folder, e.g., C:\platform-tools.
- Add C:\platform-tools to your PATH environment variable.

Verify from a new Command Prompt:

```
adb version
```

Step 4 — Install Python 3 and Git

- Download and install Python 3 from python.org (check “Add python.exe to PATH” during setup).
- Download and install Git for Windows from git-scm.com.

Step 5 — Install ALEAPP

```
git clone https://github.com/abrignoni/ALEAPP.git
cd ALEAPP
py -m pip install -r requirements.txt
```

Step 6 — Install ExifTool

- Download the Windows executable from exiftool.org, rename it exiftool.exe, and place it in a folder that is on your PATH (e.g., alongside platform-tools).

Version Note: Package/version numbers change over time. If a command in this manual fails, check the tool's --help or -h output first — flags occasionally change between releases.

5. Preparing the Practice Evidence Set

This section is identical on Linux and Windows — only the shell prompt differs.

Step 1 — Start your Android emulator (AVD) from the Fundamentals lab

Launch Android Studio → Device Manager → start the same AVD used previously.

Step 2 — Confirm ADB sees the emulator

```
adb devices
```

Step 3 — Pull a logical extraction of the /sdcard area (unrooted-safe)

```
mkdir extraction
adb pull /sdcard extraction/sdcard
```

Step 4 — (If your AVD is rooted / a “rooted” system image) pull app data for analysis

```
adb root
adb pull /data/data extraction/data
```

If adb root fails: Standard AVD system images support adb root because they are debug builds; production devices do not. If your image reports “adb cannot run as root,” re-create the AVD choosing a system image labeled “Google APIs” (not “Google Play”) — those are debuggable.

Step 5 — Create a raw disk image for the Sleuth Kit exercises (optional but recommended)

To also practice partition- and file-system-level analysis (not just logical file pulls), create a small virtual disk, format it, and copy some of the extracted files onto it. On Linux:

```
dd if=/dev/zero of=practice.img bs=1M count=256
mkfs.ext4 practice.img
mkdir mnt_pt && sudo mount -o loop practice.img mnt_pt
sudo cp -r extraction/sdcard/* mnt_pt/
sudo umount mnt_pt
```

On Windows, create an equivalent image using a VHD:

```
diskpart
create vdisk file=C:\lab\practice.vhd maximum=256
select vdisk file=C:\lab\practice.vhd
attach vdisk
create partition primary
format fs=ntfs quick
assign letter=P
exit
```

Then copy the extracted sdcard files into P:\ using File Explorer, and detach the VHD (Disk Management → right-click → Detach VHD) before analyzing it as an image file in Section 6.

6. Session 2 — Command-Line File-System Analysis with The Sleuth Kit

Duration: 2 hours | Same commands on Linux and Windows once TSK is on your PATH.

6.1 View the Partition Layout — mmls

```
mmls practice.img
```

Record the Start sector of the partition you created — you will pass it as an offset to every command below.

6.2 Inspect the File System — fsstat

```
fsstat -o <start_sector> practice.img
```

fsstat prints the file-system type, volume metadata, and (for journaling file systems) journal details. Record the file-system type and block size in your worksheet.

6.3 List Files, Including Deleted Ones — fls

```
fls -o <start_sector> -r practice.img
```

The -r flag recurses into subdirectories. Entries prefixed with * are marked deleted but not yet overwritten — flag at least one such entry (if present) for the next step.

6.4 Inspect a File's Metadata — istat

```
istat -o <start_sector> practice.img <inode_number>
```

istat prints the inode's MAC(B) timestamps (Modified / Accessed / Changed / Born), permissions, and the data-block addresses backing the file — the same information underlying a GUI tool's “file properties” panel.

6.5 Extract a File by Inode — icat

```
icat -o <start_sector> practice.img <inode_number> > recovered_file.bin
```

Verify the extracted file's integrity with a hash so it can be referenced reproducibly in your report:

```
sha256sum recovered_file.bin          # Linux
CertUtil -hashfile recovered_file.bin SHA256  # Windows
```

6.6 Worksheet Task

1. Run mmls, fsstat, and fls against practice.img and record the output of each.
2. Identify one file of interest from the fls listing, and record its inode number.
3. Run istat on that inode and note its timestamps.
4. Extract the file with icat and record its SHA-256 hash.

7. Session 3 — GUI Analysis, SQLite Artifacts, and Automated Parsing

Duration: 2 hours

7.1 Case Analysis with Autopsy (Windows)

8. Launch Autopsy → New Case. Enter a case name (e.g., LabSession3) and case number/examiner as prompted.
9. Add Data Source → choose Disk Image or VHD (select practice.vhd) — or choose Logical Files and point it at your extraction/sdcard folder if you skipped the image step.
10. On the Ingest Modules screen, enable at minimum: Recent Activity, Hash Lookup, File Type Identification, and Extension Mismatch Detector. Click Next, then Finish.
11. Once ingest completes, expand the Data Sources tree on the left and review: Deleted Files, File Types, and Results → Extracted Content.
12. Use the Keyword Search box (top-right) to search for a term you know exists in one of your pulled files (e.g., a filename or a word from a test note) and confirm Autopsy locates it.
13. Right-click any file of interest → Extract File(s) to export it, or Generate Report → HTML to produce a case report for your worksheet.

7.2 Querying SQLite Artifacts with DB Browser for SQLite

Android stores most user data — contacts, SMS, call logs, app state — in SQLite databases under each app's data directory. If your Step 4 pull in Section 5 succeeded, locate a database such as:

```
extraction/data/com.android.providers.telephony/databases/mmssms.db
extraction/data/com.android.providers.contacts/databases/contacts2.db
```

14. Open DB Browser for SQLite → Open Database → select the .db file (choose “All files” if it isn't listed by default).
15. Switch to the Browse Data tab and select the sms or message table (name varies by Android version).
16. Switch to the Execute SQL tab and run a targeted query, for example:

```
SELECT address, date, body FROM sms ORDER BY date DESC LIMIT 20;
```

17. Note how the date column is stored as a Unix epoch timestamp (milliseconds) — convert one value to a human-readable date and record both values in your worksheet.

7.3 Automated Parsing with ALEAPP

ALEAPP automates exactly this kind of database/plist parsing across dozens of known Android artifact types.

```
cd ALEAPP
python3 aleapp.py -t fs -i /path/to/extraction/data -o /path/to/aleapp_output
```

On Windows, replace python3 with py:

```
py aleapp.py -t fs -i C:\lab\extraction\data -o C:\lab\aleapp_output
```

18. Once processing finishes, open the generated index.html in the output folder.
19. Navigate the left-hand category list (e.g., SMS & MMS, Contacts, Installed Apps) and compare at least one artifact category against what you found manually in Section 7.2.
20. Record any artifact ALEAPP surfaced that you had not found manually, and explain why an automated parser is valuable even when analysts know how to query the raw databases themselves.

8. Lab Deliverables

- Worksheet Part 1 — mmls / fsstat / fls output, inode analysis, icat extraction and hash (Section 6.6).
- Worksheet Part 2 — Autopsy case screenshot(s) showing ingest results and one keyword-search hit, or exported HTML report (Section 7.1).
- Worksheet Part 3 — SQL query and results from DB Browser, with the epoch timestamp converted to a readable date (Section 7.2).
- Worksheet Part 4 — ALEAPP output screenshot/summary and the reflection question answer (Section 7.3).

Submit a separate PDF or Word document via the course LMS links as follows:

Part 1. as LAB 1 (Deadline 1 week)

Part 2. as LAB 2 (Deadline 1 week)

Part 3 and 4 Combined. as LAB 3 (Deadline 1 week)

9. Grading Rubric (6 points)

Criterion	Points
Session 1: LAB 1	2
Session 2 : LAB 2	2
Session 3 : LAB 3	2
Total	6

10. References

Tamma, R., Tindall, D., Mahalik, H. & Skulkin, O. (2020) Practical Mobile Forensics, 4th edn. Birmingham: Packt Publishing — Chapters on data acquisition and analysis.

Carrier, B. — The Sleuth Kit and Autopsy documentation: sleuthkit.org.

ALEAPP project documentation: github.com/abrignoni/ALEAPP.